WORKSHOP EPSI / WIS — BAC+5 — OCTOBRE 2026

Project Resilience-Alpha

Dossier d'ingénierie et de gouvernance

Détection d'anomalie et reprise automatique d'un service de paiement fictif

Groupe	**SafeFlow — équipe n° 2**
Effectif	6 personnes
Livrable	`Workshop2026-M2-G2-Dossier.pdf`
Version du code	commit `c11ec6f` (arbre de travail modifié)
Date du dossier	22 octobre 2026
Périmètre d'essai	laboratoire isolé `RESILIENCE-G2` — `192.168.137.0/24`

AUTEURS — À COMPLÉTER AVANT REMISE

Réf.	Nom et prénom	Domaine principal
Membre 1	à compléter	Paiement, registre, API
Membre 2	à compléter	Infrastructure, Compose, remédiation
Membre 3	à compléter	Observabilité, détection ML
Membre 4	à compléter	Sécurité, secrets, périmètre d'attaque
Membre 5	à compléter	Gouvernance, PCA/PRA, recette
Membre 6	à compléter	Interface, assistant documentaire

> **Nature du document.** Ce dossier décrit un prototype pédagogique. L'entreprise *ALPHA PAY*, ses comptes et ses transactions sont fictifs ; aucune donnée bancaire réelle, aucun moyen de paiement et aucun système de production ne sont impliqués. Les essais de charge ont été conduits sur un réseau de laboratoire isolé, contre le système du groupe uniquement.

Les affirmations chiffrées renvoient à un fichier de preuve horodaté du dépôt (`runtime/evidence/`). Les éléments non mesurés sont signalés « à confirmer » et ne sont jamais présentés comme des résultats.

Sommaire

1. Résumé exécutif — p. 2
2. Besoin métier et périmètre — p. 3
3. Architecture — p. 4
4. Composants et décisions techniques — p. 5
5. Scénario d'incident — p. 6
6. Détection ML — p. 7
7. Remédiation et preuve de reprise — p. 8
8. Observabilité — p. 9
9. Sécurité et gouvernance — p. 10 et 11
10. Résultats des essais — p. 12
11. PCA/PRA, limites et améliorations — p. 13
12. Contribution des six membres — p. 14
13. Annexes A à D — p. 15 et 16

1. Résumé exécutif

Objectif

Démontrer qu'un service de paiement fictif, rendu indisponible par l'abus d'une fonction légitime, peut être rétabli **sans intervention humaine**, et que ce rétablissement est **prouvé par une transaction métier** plutôt que par l'état d'un conteneur.

Scénario testé

Un client authentifié de faible privilège (`loadbot`) enchaîne, depuis un second PC, des demandes de devis valides sur `POST /api/quotes`. Chaque devis occupe 0,6 s un des **quatre** emplacements de capacité aval simulée, partagés avec les paiements : la capacité sature et les paiements de l'opérateur sur `POST /api/payments` reçoivent `503`. Les requêtes étant valides, le WAF ne les bloque pas ; la détection repose donc sur le *comportement* (métriques et journaux), pas sur une signature. La charge est bornée à 20 req/s, 12 requêtes simultanées, 700 requêtes et 35 s au maximum. Ce n'est pas une attaque distribuée : c'est un générateur de charge mono-poste, mono-identité.

Résultat principal

21 reprises complètes sont attestées par les preuves d'exécution (`runtime/evidence/acceptance-*.json`) : incident `recovered`, transaction témoin validée, total du registre exactement à 200 000 centimes et quatre étapes horodatées présentes. Le délai détection → service rétabli vaut **2,47 s au minimum, 2,82 s en médiane, 6,62 s au maximum** — toutes sous la cible interne de 60 s. L'indisponibilité réellement perçue par la sonde métier va de **0,52 s à 8,20 s** (médiane 2,07 s).

Un **mode témoin sans remédiation** a été ajouté pour mesurer l'écart. Dans ce mode, le détecteur continue de mesurer, d'alerter et de journaliser, mais le remédiateur n'applique aucun confinement pendant une fenêtre bornée. Les deux essais témoins exploitables montrent **48 et 54 paiements fictifs refusés** sur ~121 sondes, contre **12 à 16 sur ~115** en mode protégé : l'admission ciblée divise par environ quatre la proportion de paiements refusés sous la même charge.

Limites, dites franchement

- La reprise démontrée est un **retour de disponibilité**. Elle ne corrige pas une compromission, ne supprime pas l'attaquant et ne couvre pas une attaque répartie sur plusieurs identités.
- Le **rappel de détection n'est pas mesuré** : `ml-report.json` porte explicitement `attack_recall: "NOT MEASURED"`. Sur 38 observations d'essai, 14 n'ont vu aucun incident — attaque non lancée ou charge insuffisante.
- Le projet **n'est pas conforme** à NIS2 ou DORA et ne le prétend pas : ces textes ont orienté des décisions, les écarts sont listés au §9.6.
- Point de défaillance unique assumé : un seul PC Hôte, une seule base, pas de TLS applicatif.

2. Besoin métier et périmètre

2.1 Hypothèse d'activité

L'entreprise fictive *ALPHA PAY* traite des paiements pour un marchand fictif. Sa fonction prioritaire est le **traitement fiable et traçable d'une transaction**. Une interruption retarde les demandes, dégrade la confiance et crée une charge d'exploitation. Aucun chiffre financier réel n'est fourni : l'hypothèse illustrative retenue en atelier de gouvernance (10 transactions/min × marge fictive de 0,10 €, soit 1 €/min de contribution différée) sert à hiérarchiser, pas à chiffrer une perte.

2.2 Ce que « paiement » désigne ici

Le registre tient deux comptes, `alice` et `merchant`, initialisés à 100 000 centimes chacun. Les montants sont des **entiers en centimes**, de 1 à 100 000 ; il n'y a ni carte, ni IBAN, ni appel à un prestataire externe. Débit, crédit et écriture de transaction sont validés dans une seule transaction base de données, les lignes étant verrouillées dans un ordre déterministe. **Le total doit rester exactement 200 000 centimes** : c'est l'invariant que la preuve de reprise contrôle. Une clé d'idempotence identique avec le même contenu renvoie la transaction existante ; la même clé avec un contenu différent renvoie `409`.

2.3 Exigence de continuité retenue

Objectif	Définition retenue	Statut
RTO (*Recovery Time Objective*)	45 s entre la première indisponibilité observée et une transaction confirmée.	cible, voir §11
RPO sur ce scénario	Aucune transaction déjà confirmée perdue. Le confinement ne réinitialise ni compte ni donnée.	tenu : total à 200 000
RPO en cas de perte Hôte/base	Dépend de la dernière sauvegarde réellement disponible.	non résolu
Cible interne du chrono	Détection → transaction témoin validée en moins de 60 s (45 s objectif resserré).	tenu sur 21/21

RTO et chrono ne mesurent pas la même chose : le chrono part de la *détection* (heure d'émission de l'alerte par le détecteur), le RTO part de la *première indisponibilité constatée*. Les deux sont rapportés séparément au §11.

2.4 Périmètre d'essai et interdits

Dans le périmètre

- PC Hôte Windows 10/11, Docker Desktop + WSL2, l'ensemble de la pile.
- PC Attaquant : uniquement le client de charge Python, connecté au seul point d'accès du groupe.
- Cible unique : `http://192.168.137.1:8081`, le WAF, seul point d'entrée publié.
- PC IA optionnel : Ollama seul.

Hors périmètre, explicitement

- Réseau du campus, autre groupe, équipement tiers, Internet.
- Scan réseau, exploit, élévation de privilèges, persistance, exfiltration.
- Toute cible qui n'est pas une IP littérale du sous-réseau du lab : refusée *avant* connexion par `resolve_target`.

> **Garde-fou vérifié par test.** Le client de charge exige l'option `--lab-authorized` et refuse de démarrer sans elle ; il refuse un nom d'hôte, une adresse du campus ou d'Internet, et ne suit aucune redirection. Ces refus sont couverts par `tests/test_configs.py` (`test_attack_target_stays_inside_lab`, `test_attack_refuses_without_ack_or_outside_lab`), qui vérifient aussi que la demande de mot de passe n'est jamais atteinte quand le périmètre est violé.

2.5 Ce que le prototype ne prétend pas être

Ni une plateforme de paiement, ni un SIEM, ni une solution de haute disponibilité. La faiblesse exploitée — un devis lent partageant la capacité des paiements — est un **défaut de conception volontaire et documenté**, introduit pour rendre le service réellement saturable sans attaquer l'OS, la mémoire ni le réseau. Une architecture industrielle séparerait les pools devis et paiement, avec quotas, files et budgets de ressources dès la conception.

3. Architecture

Quinze conteneurs, huit réseaux Docker dont quatre strictement internes, un seul port publié. Le schéma distingue le *flux de requêtes* (traits pleins) de la *boucle de détection et de remédiation* (traits tiretés).

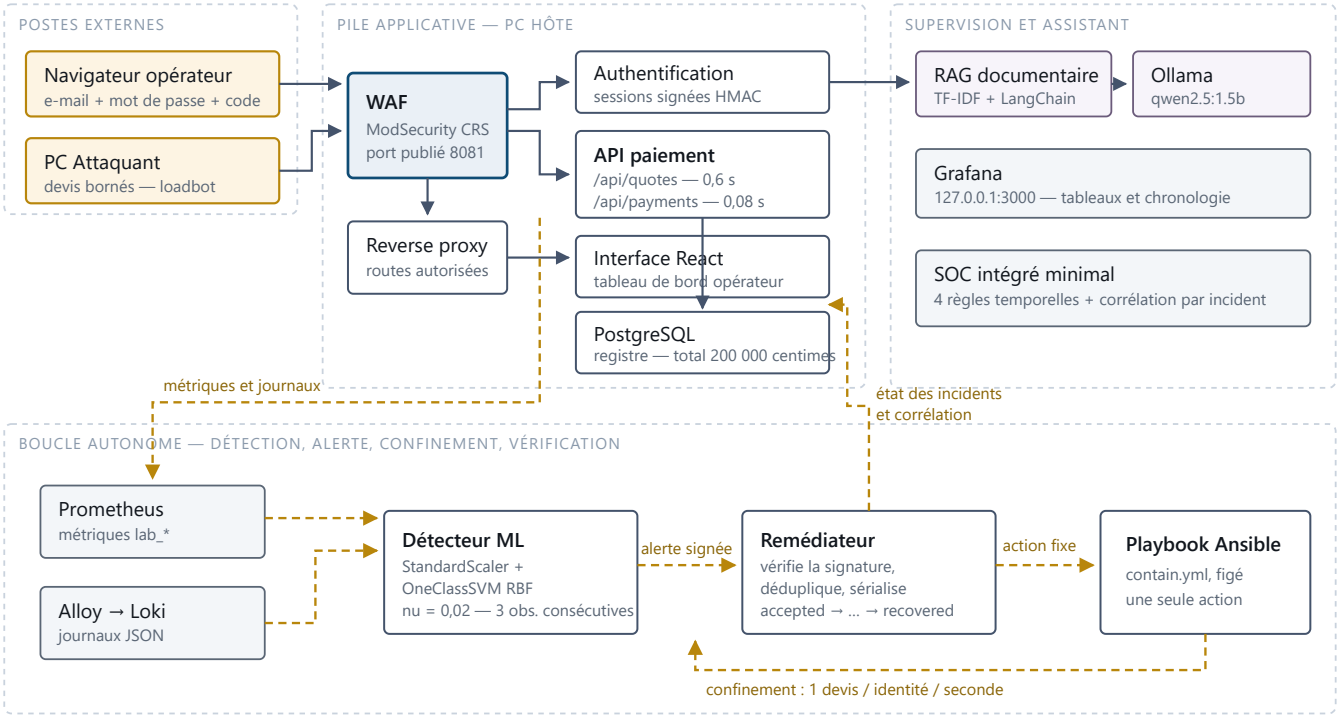


Figure 1 — Flux de requêtes (traits pleins) et boucle autonome détection → alerte → confinement → vérification (traits tiretés). Aucun socket Docker n'est confié à un service ; le RAG ne reçoit aucun secret de contrôle et n'exécute aucun outil. Source : `compose.yaml`, `infra/nginx/default.conf`, `docs/02-ARCHITECTURE.md`.

3.1 Exposition réseau

Seul le WAF est publié, sur `${LAB_BIND}:${LAB_PORT}` (8081 par défaut : 8080 est déjà pris sur le poste Hôte). Grafana reste sur `127.0.0.1:3000`. Base, Prometheus, Loki, Alloy, le détecteur, le remédiateur et Ollama **n'exposent aucun port**. Les réseaux `app`, `data`, `control` et `observe` sont déclarés `internal: true`. Les routes externes sont énumérées explicitement dans le reverse proxy ; tout le reste renvoie `404`. Il n'existe aucun chemin public vers `/control/contain` (service de paiement) ni vers `/alerts` (remédiateur).

3.2 Cloisonnement réseau

Huit réseaux Docker, dont quatre `internal: true` : ni accès sortant, ni publication possible. Un service n'est joignable que par ceux qui partagent un de ses réseaux.

Réseau	Isolé	Services attachés	Ce que le cloisonnement empêche
`front`	non	waf, edge, remédiateur	Seul `waf` y publie un port.
`app`	**oui**	auth, paiement, remédiateur, frontend, edge, rag	Le trafic applicatif ne sort jamais de l'Hôte.
`data`	**oui**	base, paiement	La base n'est joignable que par le service de paiement.
`control`	**oui**	paiement, remédiateur	L'action de confinement n'est atteignable que depuis le remédiateur.
`observe`	**oui**	paiement, remédiateur, détecteur, edge, Prometheus, Loki, Alloy, Grafana	Métriques et journaux restent internes.
`ai`, `mail`, `mgmt`	non	ollama + rag ; auth ; Grafana	IA locale et envoi du code de connexion séparés du reste ; Grafana sur `127.0.0.1`.

3.3 Chaîne visée, en une phrase

Transaction confirmée → charge de devis bornée depuis le second PC → paiements en `503` → détection comportementale sur métriques et journaux → alerte signée HMAC → playbook Ansible fixe activant une admission ciblée → transaction témoin vérifiée *à travers le WAF* → paiement réussi alors que la charge est toujours active.

4. Rôle des composants et décisions techniques

Composant	Version / image	Rôle effectif et point notable
API paiement	FastAPI 0.115.12	`POST /api/quotes` (devis, 0,6 s) et `POST /api/payments` (paiement, 0,08 s) partagent **4 emplacements** de capacité ; au-delà, `503` sans aucune écriture. `POST /control/contain` est interne, protégé par un secret dédié, et persiste la politique sur volume. Un seul worker : le compteur de capacité est en mémoire.
Authentification	FastAPI	Deux chemins. *Opérateur* : e-mail + mot de passe haché + code à 6 chiffres envoyé par courriel (expire en 5 min, 5 essais, 10 demandes/min/adresse). *Comptes de laboratoire* `alice` / `loadbot` : `POST /auth/token` , 30 connexions/min/adresse vue.
Registre	PostgreSQL 17.4-alpine	Comptes et transactions dans une transaction unique, lignes verrouillées en ordre déterministe, montants entiers. SQLite ne sert qu'aux tests natifs et **ne valide pas** PostgreSQL.
WAF	owasp/modsecurity-crs:nginx	Seul point d'entrée publié, filtrage de contenu CRS. **Aucune limitation de débit** n'y est configurée (`infra/nginx/default.conf` n'a ni `limit_req` ni `limit_conn`) : un devis valide passe CRS. Le tag `nginx` est mobile — figer son digest reste à faire.
Détecteur ML	scikit-learn 1.6.1	StandardScaler + OneClassSVM RBF (`gamma='scale'` , `nu=0,02`). Quatre entrées lues toutes les 2 s dans Prometheus et Loki. Journalise chaque observation, ce qui rend la fenêtre d'attaque relisible après coup.
Remédiateur	Ansible Core 2.18.4	Reçoit des alertes signées HMAC-SHA256, valide UUID, fraîcheur (±30 s) et nombres, déduplique durablement, sérialise la correction. Une seule action autorisée. Délai Ansible borné à 25 s. Pas de socket Docker.
Prometheus	v3.2.1	Compteurs `lab_requests_total` , `lab_capacity_denied_total` , jauges `lab_active_work` , `lab_contained` , histogramme `lab_duration_seconds` . Rétention 2 jours.
Alloy → Loki	v1.8.2 → 3.4.2	Expédie les journaux JSON métier et l'audit WAF. Rétention 48 h. Alloy tourne en root *en lecture seule* pour lire les fichiers.
Grafana	11.6.0	Tableaux de bord et panneau de chronologie d'incident. Lié à `127.0.0.1` uniquement.
RAG	LangChain Core 0.3.59	Corpus `docs/*.md` , recherche lexicale TF-IDF (3–5 caractères), 3 extraits au plus, génération par Ollama `qwen2.5:1.5b` à température 0. **Aucun outil d'action, aucun secret de contrôle.** Hors corpus : « Information absente du corpus. » *sans appeler le modèle.*
Interface	React 19 / Vite 6.2.6	Navigation par barre latérale après connexion : vue d'ensemble, service paiement, supervision, services de démonstration. Le chrono est calculé sur les horodatages du serveur, jamais sur une horloge du navigateur.

4.1 Trois décisions structurantes et leur coût

D-1 — On contient, on ne recrée pas le conteneur

Recréer un conteneur exigerait de confier le socket Docker — un pouvoir équivalent à root sur l'Hôte — à un service qui traite des entrées venues du réseau. Le remédiateur reçoit donc deux pouvoirs étroits : appeler une action fixe sur le service de paiement avec un secret dédié, et écrire une liste de bannissement. *Coût :* un conteneur réellement compromis ne serait pas remplacé ; la réponse relève alors d'une décision d'opérateur (confinement, investigation, reconstruction depuis une image saine), hors de l'automate. Le `restart: unless-stopped` de Docker n'est pas une remédiation.

D-2 — OneClassSVM plutôt qu'IsolationForest

Deux des quatre variables — rejets de capacité sur 10 s et leurs journaux — valent exactement 0 sur toute baseline valide, alors que ce sont les signaux directs de l'incident. Un IsolationForest coupe entre le minimum et le maximum appris : une variable constante à l'apprentissage ne peut pas être coupée et n'apporte rien. Avec StandardScaler, scikit-learn n'applique aucune mise à l'échelle à une variable d'écart-type nul : 30 rejets sur 10 s se retrouvent à 30 unités du nuage normal et sortent nettement de la frontière. L'apprentissage du SVM est de plus déterministe, ce qui compte pour une démo reproductible. *Coût :* les deux modèles n'ont pas été comparés sur la baseline réelle ; le choix repose sur l'analyse des données et un test unitaire, pas sur un banc d'essai.

D-3 — Jeton de session signé HMAC, pas un jeton standard

Le service d'authentification émet `base64url(charge).base64url(HMAC-SHA256)` portant identité, rôle, audience `resilience-lab` et expiration à 1 h ; chaque service le vérifie avec la même clé, un secret Docker. Une quarantaine de lignes lisibles et testées, plutôt qu'une bibliothèque de jetons dont il faut maîtriser les options en quatre jours. *Coût, réel :* la clé est symétrique et partagée, donc **tout service qui la détient peut émettre un jeton opérateur**, pas seulement le vérifier — le remédiateur et le RAG la détiennent. Ni rotation, ni révocation avant expiration. En contexte industriel : OIDC et clés asymétriques.

5. Scénario d'incident

5.1 Nature de l'abus

Abus d'une fonction légitime pour saturer une capacité partagée. Le compte `loadbot`, autorisé uniquement à demander des devis, enchaîne des requêtes *valides*. À 20 req/s, les devis réclament une douzaine d'emplacements pour quatre disponibles : la capacité reste saturée et les paiements reçoivent `503`, sans aucune ligne de débit. Ce n'est ni une compromission, ni une fuite de données, ni une élévation de privilèges.

5.2 Bornes appliquées par le code

Borne	Valeur	Où elle est appliquée
Débit	20 req/s au maximum (une toutes les 50 ms)	boucle d'envoi
Concurrence	12 requêtes en vol, 12 connexions	boucle d'envoi, `httpx.Limits`
Volume total	700 requêtes	boucle d'envoi
Durée	`--seconds` de 5 à 35, **30 par défaut**	contrôle des arguments
Cible	`POST /api/quotes`, corps `{"amount_cents": 100}`	code du script
Périmètre, accord	IP littérale de `192.168.137.0/24` ou loopback ; `--lab-authorized` obligatoire	`resolve_target`, contrôle des arguments

5.3 Symptômes HTTP observés, et ce qu'ils désignent

> **Deux séries de codes HTTP ne doivent pas être confondues.** Les codes mesurés par l'**observateur sur l'Hôte** portent sur ses propres **paiements fictifs** (`POST /api/payments`, une sonde toutes les 0,5 s) : ce sont eux, et eux seuls, que ce dossier cite au §10. Les codes du **récapitulatif du client de charge**, lui, portent sur ses **devis** (`/api/quotes`), où un `429` après confinement est *attendu*. Ce fichier reste sur le PC Attaquant et **n'a pas été rapatrié** : aucun chiffre ne lui est emprunté. à confirmer

5.4 Chronologie d'un essai protégé

Déroulé reconstitué depuis les étapes horodatées de `acceptance-1792585122.json` (essai du 21 octobre 2026, 12:17:31 UTC — le plus rapide des 21).

Décalage	Événement	Observation ou preuve
avant	Transaction initiale, puis lancement manuel de la charge	Paiement fictif `200` : service sain. La charge démarre sur le PC Attaquant au message « Start attack_lab.py ».
—	Saturation	16 paiements fictifs refusés `503` sur 115 sondes ; `lab_capacity_denied_total` et `lab_active_work` montent ; journaux `capacity_denied` dans Loki.
t_0 = 0,00 s	`detected`	Heure d'émission signée de l'alerte par le détecteur, après 3 observations anormales consécutives.
+0,02 s	`containing`	Alerte acceptée, playbook lancé. **L'alerte est prise en charge en 20 ms.**
+2,31 s	`verifying`	Playbook terminé, admission ciblée active et persistée ; healthcheck `/ready` passé.
+2,47 s	`recovered`	Transaction témoin d'un centime créée *à travers le WAF*, retrouvée au registre, total à 200 000 centimes.
après	Service rétabli sous charge	Paiement final `200` alors que la charge est toujours active ; le `loadbot` reçoit des `429`.

Le chrono part de l'heure signée `issued_at` du détecteur, pas de sa réception par le remédiateur. Le délai *charge → alerte* n'est pas isolé dans ces preuves : la seule mesure disponible est le délai calibré par rejeu (§6), ~11,7 s. à confirmer sur l'Hôte

5.5 Signaux utilisés pour décider

La décision n'utilise **aucune signature de requête**. Elle combine quatre mesures (débit de devis servis, occupation de la capacité, rejets de capacité, journaux de rejet) et exige *conjointement* un score de nouveauté négatif et la présence de rejets de capacité — une barrière d'autorisation qui évite de corriger une nouveauté inoffensive, sans remplacer le modèle ni servir à juger sa qualité (§6).

6. Détection ML

6.1 Modèle réellement implémenté

Vérifié dans `services/detector/model.py` : un pipeline scikit-learn `make_pipeline(StandardScaler(), OneClassSVM(kernel='rbf', gamma='scale', nu=nu))`. C'est un modèle de **détection de nouveauté à une seule classe** : il apprend le fonctionnement nominal, sans étiquette d'attaque. Il n'y a **ni classification binaire supervisée, ni IsolationForest, ni modèle préentraîné**. Un score négatif signale une nouveauté.

6.2 Entrées

Variable (dans l'ordre)	Origine réelle	Fenêtre
`quote_rate`	Prometheus — `lab_requests_total{route="quotes"}`	`rate` 10 s
`active_work`	Prometheus — `lab_active_work`	instantané
`capacity_denied_10s`	Prometheus — `lab_capacity_denied_total`	`increase` 10 s
`denial_logs_10s`	Loki — événements `capacity_denied` du paiement	`count_over_time` 10 s

Échantillonnage toutes les 2 secondes ; aucune donnée bancaire n'entre dans le modèle. Une série absente ou une valeur non finie est **refusée** : une panne de collecte n'est jamais remplacée par un fonctionnement normal inventé — le détecteur passe en `telemetry_or_delivery_error` et le dit. Une baseline contenant des rejets de capacité est refusée à l'entraînement.

6.3 Règle d'alerte

Une observation est *anormale* si — et seulement si — le score est négatif **et** que des rejets de capacité sont présents. Une alerte part après **3 observations anormales consécutives**, avec un refroidissement de **30 s** entre deux alertes et aucun réentraînement pendant l'incident. Une seule fonction sert au compteur d'alerte, à la route de diagnostic `GET /anomaly` et au rejeu de calibration : il n'existe qu'une définition.

6.4 Calibration — et une leçon de méthode

Baseline de **90 observations** collectées sur la pile réelle, 72 pour l'apprentissage et les 18 suivantes en validation chronologique (`ml-report.json`, SHA-256 `0f42ace4…`). Un premier balayage mené *sur ce seul holdout de 18 points* proposait `nu=0,05`, `consecutive=1` à 0 % de fausses séries. **C'était trompeur** : rejoué sur une session normale indépendante de **156 observations**, ce couple produit **9 fausses séries**. L'outil refuse désormais de conclure sous 150 observations, et ce premier fichier porte `conclusive: false`.

Couple	Fausses séries (156 obs. normales)	Scores négatifs par observation	Détecte	Délai
`nu=0,05`, `consecutive=1` — proposition trompeuse du 1er passage	9	5,8 %	oui	7,7 s
`nu=0,03`, `consecutive=3` — réglage précédent	2	15,4 %	oui	11,7 s
`nu=0,02`, `consecutive=3` — retenu et déployé	**0**	**14,1 %**	**oui**	**11,7 s**

Preuve : `calibration-20261020T065603Z.json` (`conclusive: true`, 156 observations normales hors échantillon). Les valeurs déployées sont les défauts du code (`DEFAULT_NU = 0.02`, `DEFAULT_CONSECUTIVE = 3`). Le délai va du début de la fenêtre d'attaque à l'observation qui déclenche l'alerte : il exclut la livraison de l'alerte et la remédiation.

6.5 Ce que le modèle fait sur une attaque réelle

Sur les 17 preuves de jalon M7 validées, le détecteur a journalisé entre **7 et 19 observations anormales** par fenêtre d'une cinquantaine, score minimal autour de **−0,32**. Le pic est *mesuré*, pas lu sur un écran : débit de devis de **4,88 à 6,63 req/s** contre une enveloppe normale plafonnant à **1,37 req/s**, occupation à **4,0** contre **1,0**, et **81 à 398 rejets de capacité** sur la fenêtre.

6.6 Limites du détecteur, sans détour

- **Le rappel n'est pas mesuré.** `ml-report.json` porte `attack_recall: "NOT MEASURED"` : on sait que le modèle reconnaît les observations de la fenêtre d'attaque rejouée, pas quelle fraction d'incidents il détecterait en général.
- **14,1 % des scores sont négatifs sur du trafic normal.** Aucune fausse alerte ne part, car la règle exige en plus des rejets de capacité — nuls en trafic normal — mais ce filet n'est pas une qualité du modèle. Les fenêtres se chevauchant, le holdout de 18 points reste un diagnostic : son taux (33,3 % par observation) n'est pas comparable aux « fausses séries » ci-dessus.
- **Non couverts** : attaques lentes, dérive de charge, identités multiples, compromission de la télémétrie. Deux variables sur quatre sont constantes à zéro dans une baseline saine — c'est voulu, ce sont les signaux de l'incident — et le volume de baseline reste très limité.

7. Remédiation et preuve de reprise

7.1 Étapes réellement implémentées — mode protégé

État	Ce qui se passe	Garde-fou
`accepted`	Alerte reçue sur `/alerts` (interne, non publiée). Signature HMAC-SHA256 des octets exacts vérifiée ; UUID, fraîcheur ±30 s, score fini négatif, 4 variables finies, rejets > 0.	Corps > 4096 o : `413` ; signature invalide : `403` ; contenu refusé : `422`. Déduplication durable.
`containing`	Le playbook fixe `contain.yml` active l'admission ciblée : **un devis par identité et par seconde**, persistée et idempotente. Le service n'est jamais arrêté.	Playbook, URL de contrôle et action ne viennent **jamais** du corps de l'alerte. Délai borné à 25 s ; un incident à la fois.
`verifying`	Playbook terminé, healthcheck `/ready` passé. Le bannissement optionnel, s'il est actif, est *relu dans le fichier*, pas supposé.	Le ban est au mieux : son échec ne fait jamais échouer le confinement.
`recovered`	**Une disponibilité de base ne suffit pas.** Le remédiateur crée un paiement d'un centime *à travers le WAF*, vérifie son inscription au registre et le total à 200 000 centimes.	Sans ces trois conditions, l'état ne devient pas `recovered`.
`failed`	Toute exception laisse l'incident en échec, avec `manual_review_required: true` et le type d'erreur.	Aucune boucle de commandes ouverte ; aucune fausse réussite.

Un incident interrompu est repris au redémarrage par la *même* action idempotente. Le jeton de service nécessaire à la vérification est obtenu **directement auprès du service d'authentification par le réseau interne**, non à travers le WAF : derrière le proxy, tous les clients publics partagent un budget de connexion qu'un flot pouvait épuiser — la vérification recevait alors `429` et l'incident basculait en `failed`. Corrigé et verrouillé par `tests/test_login_budget.py`, qui épuise le budget public et vérifie que la vérification métier aboutit quand même.

7.2 Mode témoin sans remédiation

Ajouté pour répondre à une question simple : *la charge a-t-elle réellement un effet quand la résolution automatique n'intervient pas ?* Le détecteur continue de mesurer, d'alerter et de journaliser — l'incident est enregistré avec sa vraie heure de détection — mais le remédiateur **n'applique ni confinement, ni bannissement, ni transaction témoin**. L'incident termine dans un état **terminal distinct,** `observed`, impossible à confondre avec `recovered`.

> **Ce n'est pas un interrupteur de sécurité.** Le WAF, la connexion opérateur, la limitation des essais de connexion et toutes les autres défenses restent actifs. Garde-fous implémentés et testés :
>
> - fenêtre **bornée dans le temps** : 5 à 300 s, plafond appliqué par le service ;
> - réactivation automatique **à l'échéance**, vérifiée au point de décision et par la boucle d'entretien, même si personne ne lit l'état ; et par le lanceur dans un `finally` — étape en échec, interruption clavier ou plantage ;
> - **jamais écrite sur disque** : tout redémarrage du remédiateur revient avec la remédiation active ;
> - **opérateur authentifié obligatoire** pour armer comme pour arrêter ; confirmation explicite ; refus si une remédiation est déjà en cours ;
> - état publié à chaque interrogation de `/ops/status` : bandeau d'avertissement **sur toutes les pages** du tableau de bord, avec réactivation immédiate.

Le récapitulatif JSON consigne le mode, la fenêtre, l'impact observé et les contrôles attendus en échec. **Un essai témoin n'est jamais** `PASS` : il porte un statut propre — `CONTROL_IMPACT_OBSERVED`, `CONTROL_NO_IMPACT` ou `CONTROL_NOT_INHIBITED`.

7.3 Pourquoi deux contrôles échouent volontairement en mode témoin

`acceptance.py` et `check_mercredi.py` vérifient une **reprise** — exactement ce que le mode témoin inhibe. Leur `FAIL` est donc attendu, et listé dans `expected_failures` du récapitulatif ; `check_mardi.py`, qui mesure la charge et la détection, doit lui *réussir*. **Les critères** `PASS` **de ces trois scripts n'ont pas été modifiés** : seule la lecture de leur résultat dépend du mode.

7.4 Ce que la preuve de reprise ne couvre pas

- Le bannissement d'adresse est une **surcouche optionnelle** (`compose.ban.yaml`). Le dépôt prouve la *décision* — filtrage au sous-réseau du lab, exclusion de l'Hôte et des adresses internes, expiration à 5 min, idempotence, transaction témoin réussie avec un ban actif — mais **pas son application au WAF**, que personne n'a pu exécuter. Sous Docker Desktop, le WAF peut ne voir que l'adresse de la passerelle, hors périmètre : aucun ban n'est alors posé. à confirmer en exécution
- La vérification ne teste plus la *connexion* à travers le WAF, seulement le paiement.
- Le remédiateur utilise le compte opérateur `alice` et détient déjà la clé de session ; une identité de service dédiée au rôle minimal serait plus propre.

8. Observabilité

8.1 Ce que Prometheus mesure

Série	Type	Signification
`lab_requests_total{route,status}`	compteur	Réponses métier par route et par code. Les séries sont pré-touchées au démarrage : sans cela, après un redémarrage, la requête du détecteur ne renvoie rien et il se déclare en erreur de télémétrie.
`lab_capacity_denied_total`	compteur	Requêtes rejetées parce que la capacité est épuisée. Signal direct de l'incident.
`lab_active_work`	jauge	Emplacements de capacité aval simulée occupés (0 à 4).
`lab_contained`	jauge	1 quand l'admission ciblée est active. Permet de distinguer « avant » et « après » confinement sur un graphique.
`lab_duration_seconds{route}`	histogramme	Temps de traitement par route.

8.2 Ce que le détecteur consigne lui-même

Chaque observation scorée est journalisée comme un événement `inference` portant le score, les quatre variables et le compteur d'alerte. **C'est ce qui rend une fenêtre d'attaque relisible après coup, exactement comme le détecteur l'a vue** : `services.detector.window` l'exporte en jeu de données pour la calibration, ou en résume la fenêtre pour le contrôle de jalon. Les alertes livrées sont journalisées séparément (`alert_delivered`) avec l'identifiant d'incident.

8.3 Grafana et le SOC intégré

Grafana, lié à `127.0.0.1` , présente les séries ci-dessus et un panneau de chronologie d'incident. La fonction SOC est **minimale et assumée comme telle** : quatre règles temporelles sur Loki (au moins 5 échecs de connexion par minute, pression de capacité, livraison d'une alerte ML, activité d'audit WAF) et *une* corrélation qui suit un incident à travers les services par son `incident_id` .

Cette corrélation apporte ce qu'un simple comptage ne voit pas : une alerte **jamais traitée** (silence du remédiateur au-delà de 15 s), une reprise **bloquée** (aucun état final au-delà de 90 s), une reprise déclarée **sans toutes ses étapes** ou dans un ordre incohérent. Son état est journalisé à chaque changement, exposé par `/ops/ml` et affiché sous le chrono du tableau de bord. Ce n'est ni un SIEM, ni un SOC industriel : elle ne suit qu'un seul type d'incident, et l'activité WAF est un signal à examiner, pas la preuve qu'une ligne correspond à une attaque.

8.4 Journaux et fichiers de preuve

Dans Loki (48 h)

- Journaux JSON métier : heure, composant, événement, identifiants fictifs.
- Audit WAF au format AFZ : métadonnées et en-têtes de *réponse*.

Ni corps de connexion, ni jeton, ni clé, ni mot de passe, ni requête SQL contenant un secret. Le remédiateur, lui, ne journalise jamais le bearer qu'il manipule.

Dans `runtime/evidence/`

- `acceptance-<epoch>.json` — observations brutes et bloc `summary` .
- `m7-proof-<UTC>.json` — pic mesuré et verdict du détecteur, *y compris en cas d'échec*.
- `recette-run-<epoch>-<id>.json` — récapitulatif d'un essai, avec son mode.
- `calibration-<UTC>.json` , `ml-report.json` , `ml-baseline.json` .

> **Un échec documenté vaut mieux qu'une absence de trace.** Les contrôles de jalon écrivent leur preuve même quand ils échouent (`passed: false` et la raison). Sur les 22 preuves M7 du dépôt, **5 portent un échec** : quatre « aucun rejet de capacité sur la fenêtre » — l'attaque n'avait pas saturé le service ou la fenêtre était mal placée — dont trois avec « aucune observation anormale ». Ces fichiers sont conservés tels quels.

8.5 Limites de l'observabilité

- Rétention courte : Prometheus 2 jours, Loki 48 h. Une fenêtre plus ancienne n'est plus interrogeable, et le contrôle de jalon le signale explicitement.
- Les journaux métier JSON sur volume **ne tournent pas automatiquement** dans cette version : il faut les archiver ou les purger entre essais maîtrisés.
- La machine Hôte peut altérer ses propres journaux : **aucune preuve d'inaltérabilité** industrielle n'est apportée.
- Les horloges de Windows et des conteneurs diffèrent ; les fenêtres s'expriment en epoch ou en ISO 8601 avec fuseau, et une heure sans fuseau est refusée. Le contrôle de jalon rapporte l'écart d'horloge constaté.

9. Sécurité et gouvernance

9.1 Authentification opérateur et double facteur

La connexion au tableau de bord se fait en deux étapes. D'abord e-mail et mot de passe : l'adresse attendue est comparée en temps constant, le mot de passe vérifié contre une **empreinte**, jamais contre une valeur en clair. Ensuite un **code à six chiffres** tiré par un générateur cryptographique et envoyé par courriel à cette seule adresse. Le serveur ne conserve que l'empreinte HMAC du code, avec expiration à **5 minutes** et au plus **5 tentatives** avant destruction du défi ; les demandes sont limitées à 10 par minute et par adresse source. Les comptes de laboratoire `alice` et `loadbot` conservent un chemin direct par mot de passe, réservé aux scripts.

C'est une double authentification fonctionnelle, dépendante d'un service de courriel externe : son indisponibilité renvoie un `503` explicite et le défi est détruit. Ce n'est pas un IAM d'entreprise, et le facteur « courriel » est plus faible qu'une application d'authentification ou une clé matérielle.

9.2 Gestion des secrets

Les valeurs sont chiffrées avec SOPS et un destinataire public *age*. La clé publique chiffre ; la clé privée ne sert qu'au déchiffrement et **ne doit jamais être transmise**. Les fichiers déchiffrés nécessaires à l'exécution vivent sous un dossier en `0700`, montés en secrets Docker. **Aucune valeur n'est dans le code, dans un argument de ligne de commande, ni dans le dépôt.** Les scripts qui ont besoin d'un mot de passe reçoivent le *chemin d'un répertoire*, jamais la valeur — une valeur passée par variable d'environnement apparaîtrait dans la liste des processus. Des tests d'hygiène vérifient en continu qu'un seul utilitaire écrit une valeur de secret, que seuls deux scripts l'appellent, et qu'aucun fichier de secret n'est suivi par git.

9.3 Durcissement et exposition

Les cinq services Python tournent en non-root, racine en lecture seule, capabilities retirées, `no-new-privileges`, RAM limitée. **Aucun montage de `docker.sock`.** Les images tierces ne sont pas décrites comme intégralement durcies : certaines exigent un système de fichiers inscriptible, et Alloy s'exécute en root — en lecture seule — pour lire les journaux.

> **Une conséquence mesurée du durcissement, qui aurait coûté la démonstration.** La racine du remédiateur étant en lecture seule et Ansible résolvant `~` par la base *passwd* plutôt que par `$HOME`, ses répertoires temporaires doivent être redirigés sous `/tmp`. Sans ces trois variables, **tout module Ansible échoue** et chaque incident finirait en `manual_review_required`. Un `--syntax-check` ne détecte pas ce défaut ; seule l'exécution d'un module le révèle. La configuration est figée par `tests/test_configs.py`.

9.4 Où se trouve réellement la limitation de débit

État constaté, sans embellissement : **il n'y a aucune limitation de débit au niveau du WAF ni du reverse proxy.** Les limites sont applicatives — 30 connexions par minute et par adresse vue côté authentification, 4 emplacements de capacité partagés côté paiement (c'est de la capacité, pas une limite par client), et un devis par identité et par seconde *après* remédiation. Conséquence : le WAF ne ralentit pas un flot de requêtes *valides* ; c'est le contrôle d'admission applicatif qui le fait, une fois l'incident détecté.

9.5 Risques et traitement — synthèse EBIOS RM

Risque	V×G initial	Traitement et preuve	Résiduel
Indisponibilité devis/paiement	4×3	Détection ML, admission ciblée, transaction de vérification	2×3
Double débit lors d'un rejeu	3×4	Transaction base, verrou ordonné, idempotence ; test concurrent 40 appels	1×4
Abus de remédiation	3×4	HMAC, fraîcheur, action fixe, aucun socket Docker	1×4
Secrets dans les livrables	3×4	SOPS/age, exclusion de l'archive, revue de manifeste	1×4
Instruction malveillante via le RAG	3×3	Corpus contrôlé, aucun outil d'action, aucun secret de contrôle	2×2
Perte du PC Hôte	2×4	Sauvegarde avec empreinte ; copie externe chiffrée **à organiser**	**2×4**
Journaux ou ML indisponibles	2×3	État d'erreur explicite ; aucune fausse reprise	**2×3**

Échelle qualitative 1 (faible) à 4 (critique). Le produit V×G hiérarchise en interne, ce **n'est pas un score EBIOS officiel**. Les deux lignes grisées sont des risques explicitement *non résolus* par la démonstration. Responsables, échéances et acceptation formelle du risque restent à compléter.

9 (suite) — Gouvernance du risque

9.6 NIS2 et DORA : comment ces cadres ont orienté les décisions

> **Ce prototype n'est pas conforme à NIS2 ou DORA, et ne peut pas l'être.** L'entreprise est fictive : aucune qualification réglementaire réelle n'est possible, et NIS2 (art. 4) prévoit le rôle des textes sectoriels — les obligations des deux textes ne s'additionnent pas mécaniquement. La formulation retenue est : *démonstrateur aligné sur des thèmes de résilience, avec écarts identifiés.*

Thème	Ce que le prototype apporte	Écart industriel
Gouvernance du risque TIC	EBIOS synthétique, un responsable par domaine, décisions tracées avec leur coût	Validation direction, politique formelle, contrôles indépendants
Gestion d'incident	Détection, chronologie horodatée, action autorisée, états `recovered` / `failed` distincts	Qualification réglementaire, procédure formelle de notification
Continuité et restauration	Objectifs RTO/RPO écrits, sauvegarde et contrôle d'empreinte	Sites indépendants, exercices réguliers, sauvegardes hors machine testées
Tests de résilience	Scénario borné, 407 tests automatisés, preuves de reprise conservées	Programme de tests proportionné aux exigences applicables
Accès et secrets	Sessions signées, rôles, SOPS/age, double authentification opérateur	IAM complet, rotation et revue des droits, TLS
Prestataires TIC	Inventaire des images, du modèle et des dépendances	Diligence, contrats, registre réglementaire, stratégie de sortie
Signalement	Journal d'incident pédagogique	**Aucun signalement réel** ; rôles, seuils et délais à valider

Références consultées le 19/10/2026 : directive (UE) 2022/2555, art. 4, 20, 21, 23 ; règlement (UE) 2022/2554, applicable depuis le 17 janvier 2025 ; méthode EBIOS RM (ANSSI).

9.7 Points de sécurité que le dépôt ne vérifie pas

Trois contrôles sont écrits et documentés, mais n'ont pas été exécutés. Les citer est préférable à les laisser croire acquis.

Contrôle prévu	Ce qui est attendu	État
Filtrage WAF, testé séparément de la reprise	Un marqueur de test XSS dans une requête, non exécuté par l'application, doit être rejeté par CRS (souvent `403`) et tracé dans l'audit Loki. **Si la requête passe, le WAF ne doit pas être déclaré validé**, et il ne doit jamais être désactivé pour faire passer le scénario.	à exécuter
Bannissement temporaire d'adresse	Quatre vérifications en exécution : journal d'accès du WAF lisible, adresse du PC Attaquant visible dans Loki, `ban.applied` publié avec transaction témoin réussie, liste vidée après 5 minutes.	à exécuter
Chaîne d'approvisionnement	Relever puis figer les digests d'images après le premier téléchargement réussi, et passer un scan de vulnérabilités. Le tag du WAF est mobile ; aucun digest n'est inventé dans cette livraison.	non fait

Un quatrième point est structurel et ne se « vérifie » pas : la machine Hôte peut altérer ses propres journaux. Aucune preuve d'inaltérabilité industrielle n'est apportée, et aucune ne peut l'être dans cette architecture.

10. Résultats des essais

Essais conduits sur le laboratoire réel, l'attaque lancée à la main depuis le PC Attaquant. L'observateur sonde `POST /api/payments` toutes les 0,5 s : les codes cités sont ceux de **paiements fictifs**, jamais ceux des devis du client de charge.

10.1 Essais de reprise protégée

Preuve	Date UTC	503 / sondes	Détect.→ rétabli	Indispo. perçue	Conclusion réellement permise
`acceptance-1792584548`	21/10 12:07	13 / 115	2,79 s	1,03 s	`témoin validé` Reprise complète : 4 étapes horodatées, total à 200 000.
`acceptance-1792584771`	21/10 12:11	12 / 115	2,71 s	3,10 s	`témoin validé` Essai répété à l'identique.
`acceptance-1792585122`	21/10 12:17	16 / 115	2,47 s	2,58 s	`témoin validé` Le plus rapide des 21 ; chronologie au §5.4.
Agrégat des 21 reprises	19→21/10	12 à 19	**2,47 2,82 6,62 s**	**0,52 2,07 8,20 s**	**Reprise autonome reproductible, 21/21 sous la cible de 60 s.** Minimum, médiane, maximum.
`recette-run-1792575980`	21/10 09:46	0 / 99	—	—	FAIL Aucun incident : l'attaque n'a pas saturé. **Ne prouve rien.**
`recette-run-1792576538`	21/10 09:55	0 / 98	—	—	`NO_INCIDENT` Observation calme menée à son terme ; reprise non testée.

Agrégat calculé sur les 21 preuves où l'incident est `recovered`, le témoin `validated`, le total exactement 200 000 centimes et les quatre étapes présentes (`complete: true`). Sur les 38 preuves du dépôt, **14 n'ont vu aucun incident** : attaque non lancée, interrompue, ou charge insuffisante. Elles sont conservées et comptées.

10.2 Essais témoins sans remédiation

Récapitulatif	Date UTC	503 / sondes	Statut	Conclusion réellement permise
`recette-run-1792653302`	22/10 07:15	**54 / 121**	`CONTROL_IMPACT_OBSERVED`	Deux incidents, tous deux `observed` : aucune remédiation. 45 % des paiements refusés. **Impact constaté sans remédiation — pas une reprise.**
`recette-run-1792590023`	21/10 13:40	**48 / 121**	`CONTROL_IMPACT_OBSERVED`	Essai répété : 40 % des paiements refusés ; `check_mardi` passe, la charge est bien mesurée.
`recette-run-1792589548`	21/10 13:32	54 / 122	`CONTROL_NOT_INHIBITED`	**Essai invalide, conservé comme tel.** Deux incidents : l'un `observed`, l'autre `recovered`.
`recette-run-1792589278`	21/10 13:27	0 / 113	`CONTROL_NO_IMPACT`	Aucun rejet de capacité, aucun incident. **N'établit aucun impact.**

> **Lecture du 3e essai, et correction apportée.** `acceptance.py` y a rendu `passed: true`, parce qu'un incident `recovered` existait bel et bien. **Ce « succès » ne doit être lu ni comme une reprise démontrée, ni comme un essai témoin valide** — c'est la confusion que `CONTROL_NOT_INHIBITED` sert à empêcher. La fenêtre se refermait alors sur le premier incident et l'alerte suivante était remédiée normalement. La sémantique a depuis été corrigée : la fenêtre reste ouverte toute sa durée et consigne *tous* les incidents qu'elle couvre — les essais du 21/10 13:40 et du 22/10 le confirment.

10.3 L'écart entre les deux modes

Grandeur comparée	Mode protégé	Mode témoin	Lecture
Paiements fictifs refusés	12 à 19 / ~115 (10 à 17 %)	48 et 54 / ~121 (40 et 45 %)	Sous charge comparable, l'admission ciblée divise par ~4 la proportion de paiements refusés.
État final de l'incident	`recovered`	`observed`	Deux états terminaux distincts, impossibles à confondre.
Transaction témoin	validée (21/21)	absente	Aucune reprise n'est prouvée en mode témoin, et aucune n'est revendiquée.
Total du registre	200 000 c.	200 000 c.	L'intégrité tient dans les deux modes : la charge dégrade la disponibilité, pas les données.

10.4 Contrôles automatisés

Au 22 octobre 2026 : **352 tests Python** et **55 tests de modèle frontend** réussis, construction Vite réussie. **Deux échecs connus** subsistent et ne sont pas masqués : la chaîne d'intégration native échoue dans l'environnement de rédaction, où Ansible ne peut pas s'exécuter, et deux routes d'authentification du code ne sont pas encore décrites dans leur contrat OpenAPI. `à corriger avant remise`

11. PCA/PRA, limites et améliorations

11.1 Écart aux objectifs

Objectif	Cible	Mesuré	Écart
Détection → service rétabli cible resserrée : 45 s	< 60 s	2,47 à 6,62 s	tenu, large marge sur 21 essais, y compris sous la cible resserrée
Délai de détection depuis le début de la charge	< 10 s	~11,7 s rejeu calibré	dépassé de ~1,7 s ; non isolé sur l'Hôte
RTO : 1re indisponibilité → transaction confirmée	< 45 s	0,52 à 8,20 s indispo. perçue	cohérent, non équivalent — voir ci-dessous
RPO transaction confirmée / perte Hôte	0 perdue	0	tenu (total à 200 000) ; non mesuré pour la perte Hôte, la restauration n'ayant jamais été exécutée

> **Pourquoi le RTO n'est pas déclaré « atteint ».** L'indisponibilité perçue est mesurée par une sonde toutes les 0,5 s, entre le premier `503` et le premier `200` qui le suit. C'est une borne honnête, mais elle n'est pas identique à la définition du RTO retenue au §2.3, et le remédiateur peut avoir rétabli le service entre deux sondes. Le RTO est donc rapporté comme **cohérent avec la cible**, non comme formellement mesuré. à confirmer par une mesure dédiée

11.2 Dépendances et points de défaillance

- **Un seul PC Hôte, une seule base** : pas de haute disponibilité physique. Si Docker Desktop ou Windows s'arrête, il n'y a aucune continuité de service. Le service de paiement exige en outre **un seul worker**, le contrôle de capacité étant en mémoire.
- **Pas de TLS applicatif**, et clé de session **symétrique et partagée** (§4, D-3) : usage limité au point d'accès privé du groupe, aucun secret d'entreprise. TLS, OIDC et cycle de vie des certificats sont des écarts à résoudre avant tout usage hors prototype.
- Le budget de connexion public étant partagé derrière le proxy, un attaquant peut encore **verrouiller la connexion des opérateurs** en le saturant : se connecter avant la démonstration. La sauvegarde reste sur l'Hôte, **sa perte n'est pas couverte** tant qu'une copie externe chiffrée n'est pas organisée.

11.3 Faux positifs, non détectés

Faux positifs. Aucune fausse alerte n'a été produite : sur 156 observations de trafic normal indépendant, le couple déployé donne **zéro fausse série**. Mais 14,1 % des scores individuels y sont négatifs, et la barrière « rejets de capacité » y contribue sans être une qualité du modèle.

Non détectés. Sur 38 observations, **14 n'ont vu aucun incident** — ce n'est pas un taux de non-détection : l'attaque n'y avait pas saturé le service (quatre preuves M7 le confirment) et il n'y avait rien à détecter. **Le vrai taux de non-détection reste inconnu**, et c'est la limite la plus importante de l'évaluation : `attack_recall` vaut explicitement `NOT MEASURED`. Sont aussi mal couvertes les attaques lentes, la dérive de charge, les identités multiples et la compromission de la télémétrie.

11.4 Tests de résilience restant à mener

Incident principal répété trois fois de suite ; redémarrage du paiement avec politique conservée ; redémarrage du remédiateur *pendant* `containing` ; indisponibilité de Loki ou de Prometheus ; clé d'alerte invalide ; base indisponible ; restauration d'une sauvegarde sur une base de test. Les quatre vérifications du bannissement d'adresse sont aussi à exécuter — ou la surcouche reste désactivée, ce dont rien d'autre ne dépend.

11.5 Améliorations réalistes, par ordre de valeur

Amélioration	Pourquoi, et ce qu'elle change	Effort
Séparer les pools devis et paiement	Supprime la cause racine : un devis ne peut plus consommer la capacité d'un paiement. Quotas, files et budgets par fonction métier.	moyen
Mesurer le rappel sur plusieurs incidents	Transforme « le modèle a détecté cet essai » en mesure défendable. Dix essais suffiraient à donner un intervalle.	faible
Copie de sauvegarde externe chiffrée	Ferme le seul risque explicitement non résolu de la matrice (perte de l'Hôte).	faible
TLS applicatif et OIDC à clés asymétriques	Retire la clé symétrique partagée : seul l'émetteur peut signer. Prérequis de tout usage hors laboratoire.	élevé
Limitation de débit au WAF ; digests d'images figés	Ralentir un flot de requêtes valides avant saturation applicative ; maîtriser la chaîne d'approvisionnement, dont le tag mobile du WAF.	faible
Identité de service dédiée au remédiateur	Lui retire le compte opérateur et la clé de session : moindre privilège réel.	moyen

12. Contribution des six membres

Le tableau ci-dessous associe chaque membre au domaine effectivement couvert par le dépôt et nomme l'élément démontrable correspondant. **Les noms et la répartition nominative restent à compléter et à valider par l'équipe** : le dépôt n'enregistre pas qui a écrit quoi, et aucune attribution individuelle n'est inventée ici.

Membre	Nom	Domaine	Tâches couvertes par le dépôt	Élément démontrable
Membre 1	à compléter	Paiement, registre, API	Registre atomique, idempotence, capacité partagée volontaire, contrôle d'admission applicatif, contrats OpenAPI du service de paiement.	Total du registre à 200 000 sur les 21 reprises ; test de concurrence 40 appels même clé.
Membre 2	à compléter	Infrastructure et remédiation	Compose 15 services et 8 réseaux, durcissement, playbook Ansible fixe, machine à états du remédiateur, reprise d'incident interrompu.	Étapes `containing` → `verifying` → `recovered` horodatées ; alerte prise en charge en 20 ms.
Membre 3	à compléter	Observabilité et détection ML	Métriques et journaux, collecte de baseline, entraînement OneClassSVM, calibration, journal d'observations, contrôle de jalon M7.	`calibration-20261020T065603Z.json` : 0 fausse série sur 156 observations ; 17 preuves M7 validées.
Membre 4	à compléter	Sécurité, secrets, périmètre	SOPS/age, durcissement des conteneurs, WAF et routes, double authentification opérateur, bornes du client de charge, tests d'hygiène.	Aucun secret suivi par git ; refus de périmètre couverts par test ; code à 6 chiffres expirant en 5 min.
Membre 5	à compléter	Gouvernance, PCA/PRA, recette	EBIOS RM, analyse NIS2/DORA, objectifs RTO/RPO, sauvegarde et restauration, lanceur de recette, mode témoin sans remédiation.	13 récapitulatifs d'essai horodatés avec leur mode ; échecs conservés (`FAIL` , `NO_INCIDENT` , `CONTROL_NOT_INHIBITED`).
Membre 6	à compléter	Interface et assistant	Console React à barre latérale, chrono sur horloge serveur, panneau de recette, bandeau d'avertissement du mode témoin, RAG TF-IDF et abstention hors corpus.	55 tests de modèle frontend ; abstention « Information absente du corpus » sans appel au modèle.

12.1 Éléments transverses à répartir explicitement

Trois sujets traversent les domaines et méritent un responsable nommé au moment de la soutenance : la **chronologie de la démonstration** (qui lance quoi, et dans quel ordre, sur les deux postes), la **conservation des preuves** entre deux essais, et la **décision d'arrêt** si le périmètre réseau devient douteux. Le document de plan d'équipe (`docs/04-PLAN-EQUIPE.md`) attribue cinq domaines ; l'équipe comptant six personnes, la sixième attribution et l'accord de chacun sur son périmètre restent à confirmer

> **Note d'intégrité.** Ce dossier a été rédigé à partir du code et des fichiers de preuve du dépôt. Lorsqu'une information n'était pas vérifiable — répartition nominative des tâches, rappel de détection, application effective du bannissement d'adresse, récapitulatif du client de charge — elle est marquée « à compléter » ou « à confirmer » plutôt que comblée par une estimation.

Annexe A — Commandes de déploiement et d'essai

Prérequis : Docker Desktop avec WSL2, SOPS et une clé *age*. Aucune de ces commandes ne contient de secret ; aucune ne supprime de volume.

Étape	Commande
Déchiffrer et installer les secrets	`python scripts/init_secrets.py`
Contrôler la configuration	`docker compose config --quiet`
Déployer la pile	`python -m ansible.cli.playbook -i localhost, infra/ansible/deploy.yml`
Télécharger le modèle local	`docker compose exec ollama ollama pull qwen2.5:1.5b`
Jalon « pile levée », lecture seule	`python scripts/check_lundi.py`
Trafic normal pour la baseline	`python scripts/normal_traffic.py --seconds 240`
Collecter la baseline	`docker compose exec detector python -m services.detector.collect_baseline`
Entraîner le modèle	`docker compose exec detector python -m services.detector.train`
Recette protégée, de bout en bout	`python scripts/run_recette.py`
Essai témoin sans remédiation	`python scripts/run_recette.py --mode control`
Preuve du jalon de détection	`python scripts/check_mardi.py --last 120`
Preuve de la boucle complète	`python scripts/check_mercredi.py --max-seconds 60`
Charge, sur le PC Attaquant	`python attack_lab.py --lab-authorized --seconds 30`

> **Ne jamais utiliser** `docker compose down -v` pour rétablir la démonstration : cela efface le registre. Le reset documenté entre deux essais supprime *uniquement* le fichier de politique de confinement, puis redémarre le service de paiement. Les incidents historiques sont conservés ; l'observateur distingue les nouveaux.

Annexe B — Extrait de configuration : routes publiées

Extrait non sensible de `infra/nginx/default.conf`. Toute route non énumérée renvoie `404`.

Route publique	Service interne	Accès
`/auth/token`, `/auth/operator/challenge`, `/auth/operator/verify`	authentification	public
`/api/quotes`, `/api/payments`, `/api/accounts`, `/api/ready`	paiement	session requise sauf `/api/ready`
`/ops/status`	remédiateur	rôle opérateur, **GET seulement**
`/ops/control-trial`	remédiateur	rôle opérateur, GET et POST
`/ops/ml`	détecteur	rôle opérateur, via sous-requête d'autorisation
`/rag/ask`	assistant	rôle opérateur

Il n'existe **aucune** route publique vers `/control/contain` (service de paiement) ni vers `/alerts` (remédiateur) : ce sont les deux seuls points de contrôle, et ils restent sur des réseaux internes, protégés l'un par un secret dédié, l'autre par une signature HMAC.

Annexe C — Index des preuves citées

Fichier	Rôle	Ce qu'il établit
`acceptance-1792584548 / 71 / 1792585122.json`	reprise protégée	Trois reprises complètes détaillées au §10.1 ; chronologie du §5.4.
21 `acceptance-*.json` avec témoin validé	agrégat	Délais 2,47 / 2,82 / 6,62 s ; indisponibilité 0,52 / 2,07 / 8,20 s.
`recette-run-1792653302 / 1792590023`	essai témoin	48 et 54 paiements refusés sans remédiation ; incidents `observed`.
`recette-run-1792589548-4afbab60.json`	essai invalide	Remédiation appliquée malgré l'essai ; conservé comme échec.
`calibration-20261020T065603Z.json`	calibration	`nu=0,02`, `consecutive=3` : 0 fausse série sur 156 observations.
`ml-report.json`, `ml-baseline.json`	modèle	90 observations, 72/18, SHA-256 du jeu, `attack_recall: NOT MEASURED`.
17 `m7-proof-*.json` validées sur 22	détection	Pic mesuré 4,88–6,63 req/s contre 1,37 en enveloppe ; 7 à 19 observations anormales.
`evidence/pytest.txt`, `evidence/frontend-build.txt`	contrôles	Trace d'exécution de la suite de tests et de la construction de l'interface.

Avertissement sur le dossier de preuves. Il contient aussi 35 récapitulatifs `recette-run-*` d'une durée inférieure à 10 ms, écrits par la suite de tests automatisés et non par un essai réel. Ils sont identifiables à cette durée et à leurs horodatages groupés, et ne sont pas utilisés dans ce dossier. Les 13 essais réels durent de 78 à 87 s.

Annexe D — Documents de référence du dépôt

`docs/01-EXIGENCES.md` exigences et critères de preuve · `02-ARCHITECTURE.md` implantation et limites · `03-INTERFACES.md` et `docs/openapi/*.json` contrats · `04-ATTAQUE.md` scénario et bornes · `05-ML-RAG.md` modèle, calibration, décisions D-07 et D-08 · `06-SECURITE.md` périmètre, secrets, décisions 1 à 3 · `07-GOUVERNANCE.md` EBIOS, NIS2/DORA · `08-PCA-PRA.md` objectifs et procédures · `09-RECETTE-RENDU.md` checklist de recette et livrables · `INSTALL.md` installation · `VALIDATION.md` état des validations réellement exécutées.